

Project Specification: High-Frequency Limit Order Book (LOB)

Timeline: 6 Weeks (approx. 168 hours)

Primary Goal: Build a deterministic matching engine that processes >1 million orders/second with sub-microsecond latency, using lock-free structures.

1. Architectural Overview

This is a simulation of an exchange core (like NASDAQ or CME).

- **Core Constraint: Zero Dynamic Allocation** on the hot path. All memory must be pre-allocated at startup.
- **Components:**
 1. **Order Entry Gateway:** Simulates network packet parsing.
 2. **Ring Buffer:** Moves orders from Gateway to Engine (Lock-free).
 3. **Matching Engine:** The logic core (Price-Time Priority).
 4. **Market Data Publisher:** Outputs the current state of the book.

2. Phase-by-Phase Implementation Plan

Weeks 1-2: The Engine (Logic & Data Structures)

Goal: A correct engine that matches trades.

- **Tasks:**
 - Define Order struct.
 - Implement the Book: A Map of Price -> Queue<Order>.
 - *Optimization:* std::map is a Red-Black tree (slow). Implement a fixed-size array or a flat map for prices if the tick size is known.
 - Logic:
 - Limit Order: Add to book.
 - Market Order: Match against top of book immediately.
 - Cancel: Remove order by ID.
- **Concepts:** Order Types, Price-Time Priority, Spread.

Weeks 3-4: The Memory (Allocators & Pools)

Goal: Remove new, malloc, and smart pointers.

- **Tasks:**
 - Create an ObjectPool<Order> class.
 - At startup: Allocate 1,000,000 orders in a contiguous block.
 - Runtime: pool.rent() returns a pointer, pool.return() marks it free.
 - Replace all std::vector or std::list with custom fixed-size containers if possible.

- **Concepts:** Heap vs. Stack, Cache Fragmentation, Free Lists.

Week 5: The Concurrency (The Disruptor)

Goal: Thread-safe communication without Mutexes.

- **Tasks:**
 - Thread 1: "Network" (Generates random orders).
 - Thread 2: "Engine" (Matches orders).
 - **The Ring Buffer:** Implement a Single-Producer Single-Consumer (SPSC) queue.
 - Use `std::atomic<size_t>` for the head and tail indices.
 - Ensure proper `std::memory_order` (acquire/release) to prevent CPU reordering.
- **Concepts:** Lock-free programming, CAS (Compare-And-Swap), Memory Barriers.

Week 6: The Benchmark & Vis

Goal: Prove the speed.

- **Tasks:**
 - Write a benchmark harness using Google Benchmark.
 - Measure "Tick-to-Trade" latency (Time from OrderIn to TradeOut).
 - **Visualization:** Use a simple OpenGL/ImGui window to draw the "Depth Chart" (Bid/Ask walls) updating in real-time.
 - *Stretch:* Replay a real crypto L2/L3 data file (e.g., from Coinbase).
- **Concepts:** Latency Histograms, Percentiles (p50, p99, p99.9).

3. Resources & Learning Material

Guides & Papers

- [The LMAX Disruptor](#) - *The foundational paper on high-throughput queues.*
- [QuantStart: Building an HFT System](#) - *Good intro to the concepts.*
- [What Every Programmer Should Know About Memory](#) - *Ulrich Drepper. Hard read, but essential.*

Tools

- [Tracy Profiler](#) - *Use this to visualize your thread latency.*
- [Google Benchmark](#) - *For micro-benchmarking your memory pool.*

Code to Study

- [Kullervo HFT](#) - *Reference C++ implementation.*
- [Folly \(Facebook Open Source Library\)](#) - *Look at their `ProducerConsumerQueue.h`.*